



MARYMOUNT

U N I V E R S I T Y

RECEIPT SCANNER

3/12/2026

Marymount University

Dr. Natalia Bell

IT-548-A

James Green, Chris Duckers, Numi Tesfay

This Page Intentionally Left Blank

RECEIPT SCANNER

This document provides an overview of the Receipt Scanner application software requirements, design, and usage. This document is produced for IT-548-A, Marymount University, as a group project. Volume 3 of this document is produced and stored externally.

Planning Document Completion Date:

Software Requirements Completion Date:

Software Design Completion Date:

Software Implementation Completion Date:

Software Usage Guide Completion Date:

ORGANIZATION

This document is organized into the following sections:

Volume-1: Receipt Scanner Planning Document

Volume-2: Receipt Scanner Software Requirements

Volume-3: Receipt Scanner Software Design Description

Volume-4: Receipt Scanner code

Volume-5: Receipt Scanner Usage Guide

This Page Intentionally Left Blank

SOFTWARE PLANNING

RECEIPT SCANNER VOLUME 1

This Page Intentionally Left Blank

CHAPTER 1 PROJECT OVERVIEW

1.1 Purpose

The purpose of this project is to explore the use of modern software development tools and artificial intelligence techniques within a Python-based application to solve a practical real-world problem. Specifically, the system aims to demonstrate how Optical Character Recognition (OCR) and automated information extraction techniques can be used to digitize paper receipts and derive meaningful financial information from them.

The Receipt Scanner application will allow a user to upload images of receipts and automatically extract key fields such as transaction date, vendor name, price paid, and gallons purchased. The extracted data will then be organized into collections of receipts defined by the user.

Using these extracted values, the system will compute summary statistics for each collection, including the average price per gallon, the total amount spent, the number of receipts in the collection, and the date range represented by the receipts.

This project is intended as a proof-of-concept system designed to demonstrate the feasibility of combining OCR technology with simple automated parsing techniques in a Python environment. The application is not intended to serve as a production-ready financial system, but rather as a prototype illustrating how automation tools can assist small single-user businesses with basic expense analysis.

1.2 Roles

The development team consists of the following members:

James Green

Responsible for system planning, requirements development, implementation of core application functionality, and documentation of system behavior.

Chris Duckers

Responsible for system design development, review of requirements and implementation artifacts, and collaboration on application development.

Numi

Responsible for creating and maintaining documentation, presentation, and users guide.

All team members will participate in system testing, review activities, and documentation refinement throughout the project lifecycle.

1.3 Deliverables

The project will produce the following deliverables:

1.3.1 Planning Document

Describes the project objectives, development schedule, tools, and assumptions.

1.3.2 Software Requirements Specification (SRS)

Defines the functional and non-functional requirements of the Receipt Scanner system.

1.3.3 Software Design Description (SWDD)

Documents the architectural and implementation design of the application.

1.3.4 Application Implementation

A Python-based software system capable of uploading receipt images, performing OCR extraction, and displaying summary statistics.

1.3.5 User Guide

Documentation explaining how users can install, run, and operate the system.

This Page Intentionally Left Blank

CHAPTER 2 DEVELOPMENT

2.1 Assumptions

The following assumptions apply to the development and operation of the Receipt Scanner system:

- Users will provide receipt images in a clear and readable format suitable for OCR processing.
- Receipts are assumed to be standard retail receipts containing recognizable textual patterns such as vendor names, dates, and totals.
- The system will be used by a single user operating locally on their machine.
- Browser storage may impose limitations on the number of receipts that can be loaded simultaneously.
- Uploaded files will be validated to confirm they are image files before processing.
- Security concerns related to storing uploaded images locally are considered out of scope for this proof-of-concept implementation.
- Receipt collections will typically contain fewer than 100 receipts.

2.2 Tools and Languages

1. The Receipt Scanner system will be implemented entirely in Python.
2. The following technologies and tools will be used during development:
3. Programming Language
Python
4. Web Application Framework
A lightweight Python web framework (such as Flask or FastAPI) will be used to host a local web server and provide the browser-based interface.
5. OCR Processing
An open-source OCR engine such as **Tesseract OCR** will be used to extract text from uploaded receipt images.
6. Image Processing
Python libraries such as **Pillow** or **OpenCV** may be used for image preprocessing prior to OCR extraction.
7. Frontend Interface
The application will provide a simple browser-based interface allowing users to upload receipts, manage receipt collections, and view extracted data summaries.
8. Data Storage
Receipt images will be stored on the local file system, while extracted receipt metadata will be stored within the browser session for display and analysis.
9. All tools used in this project will be open source.

2.1 Version Control

Source code and documentation for the Receipt Scanner system will be maintained using a version control system such as Git. Version control will be used to track changes to the codebase, manage revisions to project documentation, and support collaborative development between team members.

This Page Intentionally Left Blank

CHAPTER 3 SCHEDULE

3.1 Project Milestones

3.2 Development and Timeline

Table 1-1 Development Schedule

Activity	Plan Start	Plan End	Acc Start	Acc End	Content	Person
Planning	3/08/2026	3/14/2026	3/11/2026		Create Planning Document	James
Planning	3/08/2026	3/14/2026			Perform Fact Finding	Chris,
Requirements	3/08/2026	3/14/2026			Create Requirements	James
Requirements	3/08/2026	3/14/2026			Release Requirements for Group Review	James
Requirements	3/08/2026	3/14/2026			Complete Review of Requirements	Chris, Numi
Requirements	3/08/2026	3/14/2026			Make Updates to Requirements	James
Requirements	3/08/2026	3/14/2026			Baseline Requirements	James
Design	3/15/2026	3/18/2026			Begin Design	Chris
Design	3/15/2026	3/18/2026			Release Design for Group Review	Chris
Design	3/15/2026	3/18/2026			Complete Review of Design	James,
Design	3/15/2026	3/18/2026			Make Updates to Design	Chris
Design	3/15/2026	3/18/2026			Baseline Design	Chris
Implementation	3/19/2026	3/24/2026			Begin Implementation	James, Chris, Numi
Implementation	3/19/2026	3/24/2026			Release Implementation Sections for Review	James, Chris
Implementation	3/19/2026	3/24/2026			Complete Review	Chris, James
Implementation	3/19/2026	3/24/2026			Make Updates	James, Chris,
Implementation	3/19/2026	3/24/2026			Baseline Implementation	Numi
User Guide	3/25/2026	3/28/2026			Begin User Guide	James
User Guide	3/25/2026	3/28/2026			Release Guide for Review	James
User Guide	3/28/2026	3/29/2026			Complete Review	Numi, Chris
User Guide	3/29/2026	03/30/2026			Make Updates	James
User Guide	3/29/2026	03/30/2026			Baseline User Guide	James

This Page Intentionally Left Blank

SOFTWARE REQUIREMENTS

RECEIPT SCANNER VOL 2

This Page Intentionally Left Blank

CHAPTER 1

INTRODUCTION

1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements of the Receipt Scanner application.

The purpose of this document is to define the behavior and capabilities of the system in a clear and structured manner so that developers and reviewers can understand the expected functionality of the application prior to implementation.

1.2 Scope

The Receipt Scanner system is a Python-based application designed to digitize receipt images and extract structured information from them using Optical Character Recognition (OCR).

The system allows a user to create named collections of receipts, upload receipt images, extract relevant transaction information, and compute summary statistics based on the extracted values.

The application will operate locally and will provide a browser-based interface for interacting with the system. The system is intended as a proof-of-concept prototype and is not designed for enterprise or multi-user deployment.

1.3 System Overview

The system provides a lightweight workflow for organizing and analyzing receipt data.

Users can create collections that represent groups of receipts. Receipts may then be uploaded individually or in batches. Each uploaded image will be processed using OCR to extract textual content. The system will then attempt to identify specific receipt fields such as vendor name, transaction date, price paid, and gallons purchased.

After extraction, the system will display the extracted information and allow the user to manually edit any fields if the automatic extraction is incorrect. Receipts flagged as problematic by the extraction process will be displayed separately and excluded from summary calculations.

The system will compute several summary statistics for each collection, including:

- Average price per gallon
- Total amount spent
- Date range of receipts
- Number of valid receipts in the collection

These summaries allow users such as small single-operator businesses to quickly analyze vehicle fuel expenses.

CHAPTER 2

FUNCTIONAL REQUIREMENTS

2.1 Upload Receipt Image

Table 2-1 Receipt Upload Requirements

FR1-1	Receipt Scanner shall allow the user to upload receipt images through a web interface.
FR1-2	Receipt Scanner shall support the following image formats ➤ JPG ➤ PNG ➤ HEIC
FR1-3	Receipt Scanner shall process each receipt individual after upload.

2.2 Store Uploaded File

Table 2-2 Receipt Storage Requirements

FR2-1	Receipt Scanner shall store uploaded receipt images on the local file system.
FR2-2	Receipt Scanner shall verify that uploaded files are valid image formats prior to storage.

2.3 Perform OCR

Table 2-3 Text Recognition Requirements

FR3-1	Receipt Scanner shall perform Optical Character Recognition on each uploaded receipt image to extract textual content.
FR3-2	Receipt Scanner shall run automatically after an image is uploaded.

2.4 Extract Receipt Fields

Table 2-4 Image Processing Requirements

FR4-1	Receipt Scanner shall attempt to identify the fields from the OCR output: ➤ Vendor name ➤ Transaction date ➤ Total price paid ➤ Gallons purchased
FR4-2	Receipt Scanner shall run automatically after an image is uploaded.

2.5 Display Extracted Results

Table 2-5 Display Requirements

FR5-1	Receipt Scanner shall display the extracted receipt information to the user within the web interface.
FR5-2	Receipt Scanner shall also display summary statistics for the receipt collection.

2.6 Allow Manual Correction

Table 2-6 Display Requirements

FR6-1	Receipt Scanner shall allow the user to manually edit extracted receipt fields.
-------	--

2.7 Save or Export Results

Table 2-7 Save and Export Requirements

FR7-1	Receipt Scanner shall retain receipt data within the application interface for the current session.
-------	--

CHAPTER 3

INTERFACE REQUIREMENTS

3.1 User Interface

Table 3-1 User Interface Requirements

IR1-1	Receipt Scanner shall provide a browser-based interface allowing users to: ➤ Create receipt collections ➤ Upload receipt images ➤ View extracted receipt data ➤ Edit extracted fields ➤ View collection summary statistics
--------------	---

3.2 Software Interfaces

Table 3-2 Software Interface Requirements

IR2-1	Receipt Scanner shall interact with an OCR engine to convert receipt images into machine-readable text.
--------------	--

This Page Intentionally Left Blank

CHAPTER 4

NON-FUNCTIONAL REQUIREMENTS

4.1 Usability

Table 4-1 Usability Requirements

NFR-1-1	Receipt Scanner shall provide a simple and intuitive web interface suitable for users with minimal technical experience.
---------	---

4.2 Reliability

Table 4-2 Reliability Requirements

NFR-2-1	Receipt Scanner shall identify receipts that fail field extraction and flag them as problematic rather than using them in summary calculations.
---------	--

4.3 Maintainability

Table 4-3 Maintainability Requirements

NFR-3-1	Receipt Scanner shall be implemented in modular Python components to allow future improvements to OCR processing or field extraction logic.
---------	--

4.1 Security

Table 4-4 Maintainability Requirements

NFR-4-1	Receipt Scanner shall validate that each uploaded image is an acceptable format.
---------	---

This Page Intentionally Left Blank

CHAPTER 5

ACADEMIC REQUIREMENTS

5.1 Assignment Requirements

Table 5-1 Assignment Requirements

AR-5-1	Receipt Scanner shall utilize Natural Language Processing, AI, or Intelligent Image Processing.
AR-5-2	Each Group Member shall participate in the coding process.

5.2 External Requirements

Table 5-1 Assignment Requirements

AER-5-1	All external libraries utilized will be specifically called out and included on the presentation
AER-5-2	Documentation for the project shall include all aspects of planning, documentation, and usability.

This Page Intentionally Left Blank

SOFTWARE DESIGN

RECEIPT SCANNER VOL 3

This Page Intentionally Left Blank

CHAPTER 1

INTRODUCTION

1.1 Purpose

This Software Design Description (SWDD) defines the internal architecture and design of the Receipt Scanner system. The purpose of this document is to describe how the system implements the functional and non-functional requirements defined in the Software Requirements Specification (SRS).

The SWDD provides a detailed description of the core software components responsible for receipt ingestion, optical character recognition (OCR), receipt field extraction, validation, and summary generation. It also defines the data structures and processing workflows used by the system.

The design emphasizes modularity and maintainability. The core processing logic is implemented as a standalone Python application layer that is independent of any specific user interface. External interfaces such as a web application may interact with this core processing engine without modifying its internal logic.

1.2 Scope

This document describes the design of the Receipt Scanner software system, which is implemented using Python. The design focuses primarily on the internal processing engine responsible for receipt analysis and data extraction.

The system supports the following primary capabilities:

- ingestion of receipt images
- OCR extraction of text from images
- parsing of receipt fields
- validation and flagging of problematic receipts
- organization of receipts into collections
- generation of summary statistics for collections

External interfaces such as browser-based frontends are treated as adapters that invoke the core Python services. The SWDD therefore focuses on the design of the internal processing components rather than the implementation of any particular user interface.

CHAPTER 2

SYSTEM DESIGN OVERVIEW

2.1 Design Objectives

The design of the Receipt Scanner system is guided by the following objectives:

2.1.1 Modularity

The system is structured as a collection of small, focused components responsible for distinct processing tasks such as OCR extraction, receipt parsing, and summary generation.

2.1.2 Interface Independence

The core processing logic is independent of any particular external interface. This allows the system to be accessed through different interfaces such as a web frontend, command-line interface, or automated batch process.

2.1.3 Maintainability

The system uses idiomatic Python programming practices and clear component boundaries to allow future improvements or modifications without requiring significant redesign.

2.1.4 Extensibility

The design allows core components such as the OCR processor or receipt parser to be replaced with alternate implementations without affecting other parts of the system.

2.1.5 Simplicity

Because the system is intended as a proof-of-concept prototype, the architecture prioritizes simplicity and clarity over complex infrastructure or enterprise-scale design patterns.

2.2 Architectural Overview

The Receipt Scanner system follows a modular layered architecture composed of the following major layers:

2.2.1 Interface Layer

External systems such as a web browser interface interact with the application through defined service interfaces.

2.2.2 Application Service Layer

Coordinates receipt ingestion, OCR processing, parsing, validation, and collection summary generation.

2.2.3 Domain Processing Layer

Contains the core logic for receipt parsing, validation, and data aggregation.

2.2.4 Infrastructure Layer

Provides services for OCR processing, image loading, and storage of receipt data.

The web interface does not directly perform any processing logic. Instead, it communicates with the application service layer, which orchestrates the processing workflow.

2.3 Architectural Style

The architecture of the Receipt Scanner system follows a modular layered architecture with service-based orchestration.

This approach separates:

- external interfaces
- application services
- domain logic
- infrastructure services

The layered architecture improves maintainability and enables independent development and testing of the core processing components.

This Page Intentionally Left Blank

CHAPTER 3

COMPONENT DESIGN

The core system is composed of several primary software components. Each component is responsible for a specific function within the receipt processing pipeline.

3.1 Receipt Ingestion Component

The Receipt Ingestion Component is responsible for receiving receipt images from external interfaces and preparing them for processing.

Responsibilities include:

- accepting uploaded image files
- validating file format
- storing receipt images in the local filesystem
- initiating the receipt processing workflow

Supported input formats include:

- JPG
- PNG
- HEIC

If a file does not match the supported image formats, the ingestion component rejects the file before processing.

- 3.2 Image Preparation Component**
- 3.3 OCR Processing Component**
- 3.4 Receipt Parsing Component**
- 3.5 Receipt Validation Component**
- 3.6 Receipt Repository Component**
- 3.7 Collection Summary Component**
- 3.8 Manual Correction Component**
- 3.9 Application Service Component**

This Page Intentionally Left Blank

CHAPTER 4 DATA DESIGN

- 4.1 Receipt Data Structure**
- 4.2 Receipt Collection Data Structure**
- 4.3 Collection Summary Data Structure**
- 4.4 Data Relationships**

This Page Intentionally Left Blank

CHAPTER 5

PROCESSING WORKFLOWS

- 5.1 Single Receipt Processing Flow**
- 5.2 Batch Processing Flow**
- 5.3 Manual Correction Flow**
- 5.4 Summary Generation Flow**

This Page Intentionally Left Blank

CHAPTER 6

ERROR HANDLING AND CONSTRAINTS

6.1 Error Handling

6.2 Design Constraints

.

This Page Intentionally Left Blank

USER GUIDE

RECEIPT SCANNER VOL 5

This Page Intentionally Left Blank

CHAPTER 1

INTRODUCTION

1.1 .

.